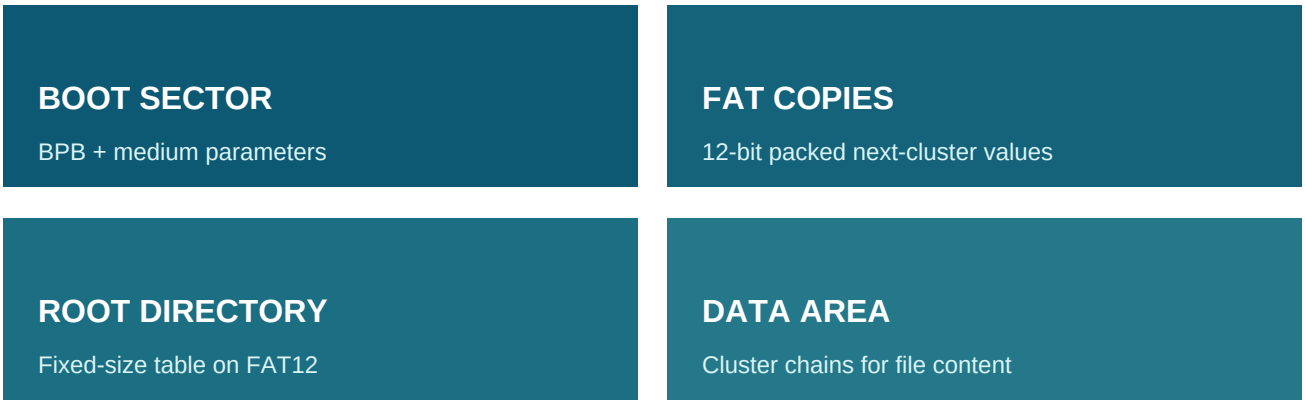
FAT12 - original disk format and history

ORIGINAL FORMAT / HISTORY / ENGINEERING DESIGN

The 12-bit File Allocation Table format, familiar from early DOS floppy disks, is a compact allocation-chain design shaped by small removable media and limited memory.

This reference describes the original upstream disk format and its historical engineering rationale. The separate InfiltratorOS adapter is not the original specification. Historical facts, format rules and implementation limitations are separated; consult the named primary sources for field-level normative requirements.

ON-DISK STRUCTURE / CONCEPTUAL MAP



CHAPTERS

1. Historical chronology
2. Original design and on-disk structures
3. Allocation and directories
4. Integrity, validation and recovery
5. Design trade-offs and historical limitations
6. InfiltratorOS interpretation

FAT12 - original disk format and history

HISTORICAL CHRONOLOGY

FAT originated during the late-1970s microcomputer era, with the early Microsoft/BASIC standalone disk systems preceding MS-DOS. FAT12 became important through IBM PC DOS and MS-DOS floppy formats in the 1980s. Small sectors, restricted RAM and removable magnetic media favoured a single compact map of cluster ownership instead of indexed inodes or complex trees. Its longevity is explained by the simple disk layout and broad boot-ROM, BIOS, DOS and cross-platform compatibility, not by transaction integrity.

The FAT family evolved through FAT12, FAT16 and FAT32. These are not arbitrary names for volume sizes: the valid data-cluster count establishes the format variant. VFAT long-name extensions later added extra directory entries without replacing the original 8.3 entries. FAT12 remained associated with floppies, embedded systems and disk images long after FAT16 and FAT32 became the norm for hard disks.

FAT12 - original disk format and history

ORIGINAL DESIGN AND ON-DISK STRUCTURES

At the start of a FAT12 volume is a boot sector and BIOS Parameter Block (BPB). The BPB records bytes per sector, sectors per cluster, number of reserved sectors, FAT copy count, root-entry capacity, media descriptor, sectors per FAT and total-sector count. The location of each subsequent region is computed from these fields. A parser must reject impossible and internally contradictory geometry rather than assume a familiar floppy layout.

Each FAT12 entry is twelve bits. Consecutive entries are packed across byte boundaries; the usual byte offset for cluster n is floor(3*n/2). An even entry uses the low twelve bits of its two-byte window, an odd entry the high twelve. Entries encode free clusters, successor clusters, reserved and bad values, and end-of-chain markers. The two initial entries are reserved and the data region starts with cluster number two.

FAT12 - original disk format and history

ALLOCATION AND DIRECTORIES

The FAT is a chain table, not an allocation bitmap with extents. A file’s first cluster is stored in its directory entry, and the FAT entry for that cluster points to the next cluster until an end marker is reached. The design keeps file entries small but reading a badly fragmented file requires a chain walk and exposes many random metadata reads. Duplicate ownership, cycles or a pointer outside the valid cluster range are corruption indicators.

The FAT12 root directory occupies a fixed region after the FAT copies; it is not a normal cluster chain. Subdirectories occupy clusters and contain entries including dot and dot-dot. Classic directory records are 32 bytes, with 8.3 names, attributes, timestamps, start-cluster and file length. VFAT long-name entries are a later compatibility layer and must be associated with the corresponding short entry using the prescribed ordering and checksum.

FAT12 - original disk format and history

INTEGRITY, VALIDATION AND RECOVERY

FAT12 has no original journal, no block checksums and no native copy-on-write transactions. Formatting traditionally creates FAT copies, but duplicated tables are not an atomic commit protocol. The dirty/clean state conventions and DOS/OS utility behaviours vary by version. Recovery must distinguish an allocation leak from cross-linked chains, lost clusters and entries with an invalid parent directory.

The correct identification sequence is: validate BPB fields, compute the data-sector area, calculate the number of data clusters, verify it is below 4085, then parse packed 12-bit entries and enforce the chosen geometry. Blindly trusting a FAT12 string label or treating a FAT16 volume as a small FAT12 volume can corrupt data. Unmounted disposable test images are required for writes, formatting and repair.

FAT12 - original disk format and history

DESIGN TRADE-OFFS AND HISTORICAL LIMITATIONS

Twelve-bit entries reduce metadata size for floppy capacities but cap the useful cluster namespace. A fixed root-directory region imposes a separate directory-entry limit and prevents normal root growth. The linked allocation table scales poorly for large or heavily fragmented files, and no permissions, resource forks, Unicode file names or crash-consistent transactions are part of the original DOS-era format.

Compatibility often matters more than speed. A modern reader must tolerate legitimate legacy BPB variants while refusing ambiguous cluster geometry. The choice of cluster size influences the maximum addressable data area and internal fragmentation. Metadata updates that span FAT copies, directory entries and user data are vulnerable to interruption.

FAT12 - original disk format and history

INFILTRATOROS INTERPRETATION

The InfiltratorOS FAT12.so module is a separate implementation of this public on-media format. Its API versioning, boot bridge and optional Linux kernel module are not properties of historical FAT12. Any published claim of write support, defragmentation, VFAT handling or recovery applies only to a tested version of our implementation.

The conceptual map on the cover is not a literal sector diagram. A forensic parser must derive actual offsets from the volume BPB, validate both packed-entry arithmetic and all cluster bounds, and preserve the source medium until independent verification has succeeded.

FAT12 - original disk format and history

PRIMARY SOURCES & DOCUMENTATION

References are included to make the historical and technical account auditable. The source documents, published format specifications and implementation references take precedence over this explanatory guide where a detail differs.

1. Microsoft FAT Specification, 2004 (original format description, accessible archived copy)

https://www.scs.stanford.edu/~zyedidia/docs/_other/fat.pdf

2. Linux kernel VFAT documentation

https://docs.kernel.org/filesystems/vfat.html

Implementation boundary: the installed InfiltratorOS .so, the optional Linux kernel .ko, and the source-first installer are distinct products. The original filesystem design is documented above; claims about the adapter’s qualification require its own source, tests and release evidence.

FAT12 / technical detail

ON-DISK FIELD AND STRUCTURE INVENTORY

Format-specific field names and meanings are summarised below; where applicable the original specification defines exact endianness, offsets, checksums and format-revision limits.

1. BPB_BytsPerSec: logical sector size, required for every offset calculation.
2. BPB_SecPerClus: power-of-two sectors in a data cluster.
3. BPB_RsvdSecCnt: extent of the region before the first FAT.
4. BPB_NumFATs: number of FAT copies, not a transactional journal.
5. BPB_RootEntCnt: fixed-size root directory capacity.
6. BPB_FATSz16: sector count of each FAT12 table.
7. Cluster entry n: floor(3*n/2) packed byte offset, selecting 12-bit half.
8. Directory entry: 32 bytes, short name, attributes, start cluster, size.

FAT12 / technical detail

HISTORICAL FORMAT DEVELOPMENT

Selected design milestones explain why this format acquired its present layout. Dates refer to historical format or implementation changes, not InfiltratorOS releases.

1. Late 1970s: early FAT designs for microcomputer disk systems.

2. 1981 onwards: PC DOS and MS-DOS spread 12-bit floppy storage.

3. 1980s: larger fixed disks drove FAT16 developments.

4. 1990s: VFAT long names added directory entry extensions without replacing FAT12.

Important historical distinction: newly introduced driver features are not necessarily changes to the original on-disk format. Read the version or feature bits rather than assuming current software support defines historic media.

FAT12 / technical detail

FORMAT VALIDATION AND RECOVERY INVARIANTS

A writable implementation must establish all of these conditions before modifying any original-format media. These checks describe the minimum class of invariants, not proof of complete recovery qualification.

1. Compute cluster count rather than trusting a type string.

2. Reject entries outside permitted 12-bit cluster limits.

3. Detect cycles, cross-linked files and orphaned allocation chains.

4. Verify both directory sizes and full cluster ownership.

5. Preserve reserved entries and media descriptor when updating FAT.

Forensic principle: use a separate verified image, retain original structures, and report conflicting evidence instead of inventing a new supposedly clean filesystem state.

FAT12 / engineering detail

DISK READ PATH / ENGINEERING NOTES

1. Interpret the volume boot record’s BIOS Parameter Block, including sector size, sectors per cluster, reserved sectors, FAT count, root entry count and FAT length.

2. Calculate data-region start from reserved sectors, FAT copies and the fixed root directory. Always add the partition start when working with whole-disk addresses.

3. Determine FAT type from the computed count of data clusters, not a human-readable FAT12 string in a boot sector or volume label.

4. Decode 12-bit FAT entries packed across byte boundaries. Odd and even cluster numbers use different nibble placement; an entry can straddle a sector.

5. Read fixed-size 32-byte directory entries, recognise deleted and end markers, and reconstruct valid long-file-name sequences only when the short-name checksum matches.

6. Translate cluster 2 to the first data cluster. Follow each subsequent FAT link until its proper end-of-chain value, checking for free, reserved and bad-cluster markers.

7. Use the file length to limit user-visible bytes from the final cluster, and do not assume every allocated cluster contains meaningful file data.

8. Account for floppy-era geometries and layout assumptions without treating physical CHS hints as the final authority for logical block placement.

FAT12 / engineering detail

VALIDATION, RECOVERY AND COMPATIBILITY

1. Cross-check cluster count against the FAT's capacity. The number of FAT sectors must accommodate the 12-bit entries and reserved initial slots.

2. Follow a file chain with visited-cluster detection, volume-range checks and a bounded maximum traversal. Circular chains must not result in endless reads.

3. Compare FAT copies as evidence; do not silently choose a damaged secondary copy as authoritative unless the operating format's rules justify it.

4. Validate the root directory's fixed allocation and ensure subdirectories are interpreted as ordinary cluster chains with dot-entry semantics where applicable.

5. Short-name aliases and VFAT long-name entries form one logical name; damaged sequence numbers or mismatching checksums must not fabricate filenames.

6. Orphaned clusters and cross-linked chains require an explicit recovery policy. A bitmap-style allocation reconstruction is not applicable to FAT12.

7. Do not mistake a volume's type label for proof that the actual cluster-count thresholds are satisfied, particularly near FAT12/FAT16 boundaries.

8. Preserve the original boot sector, FAT copies and directory entries before repair; their byte-level evidence is often more valuable than a guessed reconstructed chain.